



AT ALL NETWORKS

Python Syntax



History of Python



AT ALL NETWORKS



Python was designed as a **high-level interpreted language** focused on **simplicity, readability**, and developer **productivity** compared to more complex languages like C.

Its **technical evolution** includes early support for object-oriented programming, dynamic typing, and multiple paradigms, followed by major improvements such as list comprehensions, Unicode support, and a redesign in Python 3 for greater consistency.

Its philosophy emphasizes clear and explicit code (“**one obvious way to do it**”), supported today by a vast ecosystem of libraries for **web development, data science**, and **artificial intelligence**.

D'où vient le nom
Python ?



Program



AT ALL NETWORKS

Python syntax

Functions

Modularity

Data structures (list, dict, set, tuple)

Git & Github

Algorithms & Complexity

Problem solving





AT ALL NETWORKS

Python Syntax

Python specifics



AT ALL NETWORKS

No true access modifier

- no public/private keywords (uses naming conventions like `_` and `__`).

Dynamic typing

- variables types are determined at runtime, not declared explicitly.

Indentation defines blocks

- no `{ }`, code structure is enforced with whitespace.

Everything is an object

- even functions and classes are first-class objects.

Multi-paradigm by design

- supports object-oriented, procedural, and functional styles natively.

Duck typing

- type compatibility is based on behaviour, not explicit type.

Automatic memory management

- built-in garbage collection, no manual memory handling.

Interpreted execution

- no separate compilation step required

Built-in high-level data structures

- powerful native types like list, dict, set, tuple

Batteries-included philosophy

- large standard library available out of the box

Python syntax



AT ALL NETWORKS

Indentation

Indentation in Python is very important unlike other languages where indentation is for readability only. Indentation is used **to indicate a block of code**.

```
if 7 > 4:  
    print("Seven is greater than four!")
```



```
if 7 > 4:  
print("Seven is greater than four!")
```



Comments

```
#This is a comment
```

Begin with #

Consist of notes and reminders written by developers for other developers

Are ignored by the Python interpreter

Can be placed anywhere in code

```
'''  
~~~~~  
This is a Docstring  
~~~~~  
'''
```

Are enclosed in triple quotes (""" """)

Provide documentation for users and tools

Are stored in `.__doc__` and accessible at runtime

Are placed at the start of modules, classes, and functions

Input / Output



AT ALL NETWORKS

Output

To **display a value** in the Terminal.

```
print(*objects, sep=',', end='\n', file=None, flush=False)
```

```
print("Test 123", "Test 456", sep="/", end=" !!!\n")
```

```
Test 123/Test 456 !!!
```

Input

To **receive a value** from the user.

```
input("prompt")
```

```
name = input("What's your name ?")  
print("Hello " + name)
```



input() always return a string !

Exercice

Display "Hello world !"

Variables



AT ALL NETWORKS

Variables

To **declare** a **variable** in Python, you don't need to specify the type.

```
var_name = "value"
```

Types

You can get the data **type of a variable** with the **type()** function.

If you want to print only the name of the type, you can use **type().__name__**

```
x = 6
y = "Hello"
print(type(x))
print(type(y).__name__)
```

```
<class 'int'>
str
```

Casting

Casting can be used to **specify the data type** of a variable.

```
x = str(6)      # x is "6"
y = int(6)      # y is 6
z = float(6)    # z is 6.0
```

Exercise

Create a variable with a value given by the user and display its type.

Naming conventions



AT ALL NETWORKS

Name always written in lowercase

Words separate by underscores (_)

Cannot begin with a number

Can only contain alphanumeric characters (A-z, 0-9, _)

Must not contain accents

Case-sensitive (age, Age and AGE are different)

Cannot be one of the Python keywords (if, for, and, class, def, try, with...

Begin with underscore to indicate "internal use" (not strictly private)

Begin with double underscore (and not end with double underscore) triggers name mangling in classes

Types



AT ALL NETWORKS

Numeric types

int (arbitrary precision)

float (64-bit double)

str (text sequence)

Immutable sequences

Unicode by default

f-string formatting

bool

Represent truth values

Can be exactly True or False

subtype of int

Sequences

list

tuple

range

Sets

set

frozenset

Mapping

dict

Operators



AT ALL NETWORKS

In Python, as in other programming languages, **mathematical operators** can be used to perform various operations.

Operator	Description
+ - * /	Addition, Subtraction, Multiplication, Division
//	Whole division
%	Modulo (remainder)
**	Power

There are also, of course, the **comparison operators**.

Operator	Description
==	Equal
!=	Not equal
>, >=	Greater than, Greater than or equal to
<, <=	Lesser than, Lesser than or equal to

Operator precedence



AT ALL NETWORKS

Operator	Description
**	Exponentiation
+x, -x	Positive, negative
*, @, /, //, %	Multiplication, matrix multiplication, division, floor division, remainder
+, -	Addition and subtraction
<<, >>	Shifts (binary)
&, ^, 	Bitwise AND (1), XOR (2), OR (3)
<u>in</u>, <u>not in</u>, <u>is</u>, <u>is not</u>, <, <=, >, >=, !=, ==	Comparisons, including membership tests and identity tests
<u>not x</u>, <u>and</u>, <u>or</u>	Boolean NOT (1), AND (2), OR (3)
<u>if</u> – else	Conditional expression
<u>lambda</u>	Lambda expression
:=	Assignment expression

Exercise



AT ALL NETWORKS

Exercise

Which value return this code ?

- A. 42
- B. 48
- C. 26
- D. 50

```
result = 2 + 3 * 4 ** 2 - 8 // 3 - 6
```

Exercise

Which is the result at the end of the execution ?

- A. False
- B. True
- C. None
- D. 5

```
x = 5  
result = not x > 3 and x != 4 or x == 5
```

String



AT ALL NETWORKS

Concatenation

```
print(y + " Worlԁ!")  
print(str(1) + "Worlԁ")
```

```
Hello Worlԁ!  
1Worlԁ
```

String format

```
print(y, ", Worlԁ!")  
print(1, y, ", Worlԁ!")  
print(f"{y}, Worlԁ !")
```

```
Hello , Worlԁ!  
1 Hello , Worlԁ!  
Hello, Worlԁ !
```

Escaped Characters

Special sequences of characters in a string that would otherwise be impossible or difficult to include.

They are introduced with a \

```
print("\U0001f40d")  
print("\N{SNAKE}")  
print("🐍")
```

Escape Sequence	Meaning
\<newline>	<u>Ignored end of line</u>
\\	<u>Backslash</u>
\'	<u>Single quote</u>
\"	<u>Double quote</u>
\a	ASCII Bell (BEL)
\b	ASCII Backspace (BS)
\f	ASCII Formfeed (FF)
\n	ASCII Linefeed (LF)
\r	ASCII Carriage Return (CR)
\t	ASCII Horizontal Tab (TAB)
\v	ASCII Vertical Tab (VT)
\ooo	<u>Octal character</u>
\xhh	<u>Hexadecimal character</u>
\N{name}	<u>Named Unicode character</u>
\uxxxx	<u>Hexadecimal Unicode character</u>
\Uxxxxxxxx	<u>Hexadecimal Unicode character</u>

string manipulation



AT ALL NETWORKS

Accessing characters

A character in a string is accessed using [index].

"Character"[2] => a

```
my_string = "Python"  
print(my_string[0]) # P  
print(my_string[-1]) # n
```

Truncate a string

It is possible to truncate a string of characters using [start:end:step]

"Python"[:3] => Pyt

```
my_string = "Hello world"  
print(my_string[0:5]) # Hello
```

Exercise

Write a Python script that takes a string text and an integer max_length. If the length of text is greater than max_length, truncate it to the first max_length characters and add "..." at the end. Otherwise, keep the string unchanged.

string usefull methods



AT ALL NETWORKS

Method	Description
<code>len("string")</code>	Return the length of the string
<code>"string".upper()</code>	Converts all lowercase characters in a string to uppercase
<code>"string".lower()</code>	Converts all uppercase characters in a string to lowercase
<code>"string".split(" ")</code>	Split a string into a list of substring
<code>" ".join(iterable)</code>	Concatenate elements of an iterable (like a list, tuple or string)
<code>" string ".strip()</code>	Remove blank space before and after the string

Exercice

Write a script that takes a sentence stored in a variable text.

- Split the sentence into a list of words using the appropriate string method.
- Then, join the words back together into a new string where each word is separated by a hyphen "-".

Exercise



AT ALL NETWORKS

Exercise

Write a Python function called `truncate_string` that takes two parameters, a string `text` and an integer `max_length`.

The function should return a truncated version of the string such that:

- If the length of `text` is less than or equal to `max_length`, return the original string unchanged.
- If the length of `text` is greater than `max_length`, return a new string consisting of the first `max_length` characters followed by "...".

Truth values



AT ALL NETWORKS

Any object can be tested for **truth value** that are used in an **if** or **while** condition or as an operand of **Boolean operations**.

Operations and built-in functions with a boolean result always return :

- **0 / False** for false
- **1 / True** for true
- unless otherwise specified



Boolean operations **or** and **and** always return one of their operands.

Built-in objects considered false

Constants defined to be false

- None
- False

Zero of any numeric type

- int 0
- float 0.0
- complex 0j
- Decimal(0)
- Fraction(0)

Empty sequences and collections

- str ""
- tuple ()
- list []
- dict {}
- set set()
- range range(0)

Exercise



AT ALL NETWORKS

Exercise

What does this code print? Remember that and and or don't always return True or False – they return one of their operands.

```
a = [] or "default"
```

```
b = "hello" and 42
```

```
c = None or 0 or "fallback"
```

```
print(a, b, c)
```

Sequence types operations



AT ALL NETWORKS

Operator	Description
x in s	True if an item of s is equal to x, else False
x not in s	False if an item of s is equal to x, else True
s + t	the concatenation of s and t
s * n or n * s	equivalent to adding s to itself n times
s[i]	ith item of s, origin 0
s[i:j]	slice of s from i to j
s[i:j:k]	slice of s from i to j with step k
len(s)	length of s
min(s)	smallest item of s
max(s)	largest item of s
s.index(x[, i[, j]])	index of the first occurrence of x in s (at or after index i and before index j)
s.count(x)	total number of occurrences of x in s



AT ALL NETWORKS

Conditions

if statement



AT ALL NETWORKS

if *condition* :
 block of instructions
elif *condition* :
 block of instructions
elif *condition* :
 block of instructions
...
else :
 block of instructions

```
weather = input("What's the weather like ?")

if weather == "sunny":
    print("Put on your hat")
elif weather == "rainy":
    print("Take your umbrella")
elif weather == "cloudy":
    print("Phew, there's no rain at the moment")
else:
    print("No infos on the weather")
```

match-case statement



AT ALL NETWORKS

This is **superficially similar to a switch** statement in C, Java or JavaScript (and many other languages), but it's more similar to **pattern matching** in languages like Rust or Haskell.

- Only the first pattern that matches gets executed.
- A pattern can be a value or more than one
- A match-case can return a value.

match *status* :

case *pattern* :

 block of instructions

case *pattern* :

 block of instructions

 ...

case *_* :

 block of instructions

```
match x:
    case 0:
        print(f"You are neutral")
    case 10 | 20 | 30:
        print(f"Matched: {x}")
    case x if x < 0:
        print("You are negative.")
    case x if x in [1,2,3,4,5]:
        print("I m in the list !")
    case _:
        print("No match found")
```


Exercices



AT ALL NETWORKS

Exercice

Create a nightclub bouncer program.

Ask the user for its age.

If the user is less than 18 years old, access will be denied, otherwise they are welcome.

Exercice

Create a script that asks the user for two numbers and then displays the larger one.

Display "Tie" if there is a tie.

Bonus: Do it with three numbers.

Exercices



AT ALL NETWORKS

Exercice

Create a program that asks the user for two numbers.

If one is positive and the other negative, then the program displays "Congratulations!"

Exercice

Create a program that tells the user whether it is cold or not.

Ask the user for the temperature:

If the temperature is less than 10°C, then it is cold.

If the temperature is between 10 and 15°C, then it is cool.

If the temperature is between 15 and 20°C, then it is warm.

If the temperature is above 20°C, then it is hot.



AT ALL NETWORKS

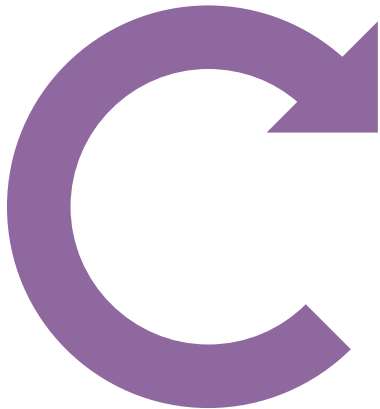
Loops

for i in range()



AT ALL NETWORKS

The "**for i in range()**" loop allows a set of instructions to be repeated a number of times defined by the "range()" function.



```
for i in range(10):  
    print("Bonjour")
```



The "range()" function returns a sequence of numbers starting from 0 (default), which increments by 1 (default) and goes up to the given number.

Exercice

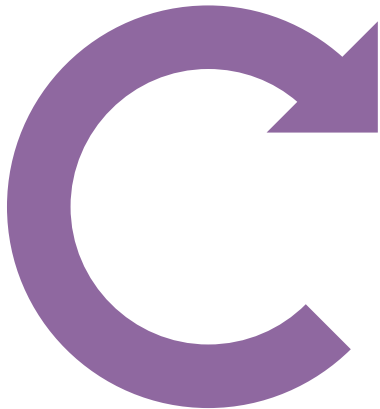
Create a loop to display "Goodbye" and the index 5 times.

for i in range()



AT ALL NETWORKS

It is possible in the "**range()**" function to specify the starting number and the number of increments in the sequence.



```
for i in range(10):
```

Starts at index 0 and goes up to index 9 in steps of 1

```
for i in range(1,10):
```

Starts at index 1 and goes up to index 9 in steps of 1

```
for i in range(1,10,2):
```

Starts at index 1 and goes up to index 9 in steps of 2

Exercise

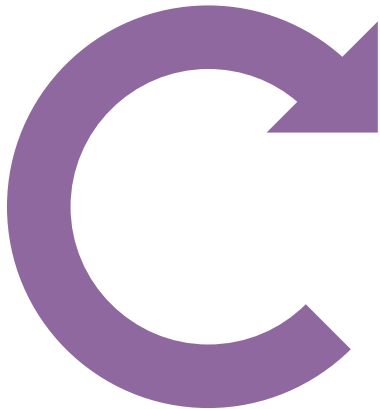
Create a loop to go from 1 to 10. For each iteration check if the index is an even number. If its even, add it to a variable called total. Display the total at the end.

for i in iterable



AT ALL NETWORKS

The "**for i in iterable**" loop over an iterable and get the element.



```
for i in "python":  
    print(i)
```

```
for i in liste_courses:  
    print(i)
```

p
y
t
h
o
n

Exercise

Use a for loop to go through a sentence. For each character :

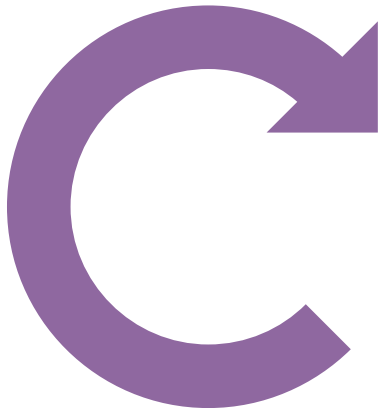
- If it is a vowel (a, e, i, o, u), print it in uppercase.
- If it is a consonant (a letter but not a vowel), print it in lowercase.
- If it is not a letter (like a space or punctuation), print "_" instead.

for i in enumerate



AT ALL NETWORKS

The "**for i in enumerate(iterable)**" loop over an iterable and get both the index and the element at the same time. It returns an enumerate object that produces pairs in the form (index, element).



```
a = ["Python", "Java", "C++"]  
for i, v in enumerate(a):  
    print(i, v)
```

```
0 Python  
1 Java  
2 C++
```

Exercise

Use a for loop to go through a list of fruits. For each element, display its index (starting from 1) and the element.

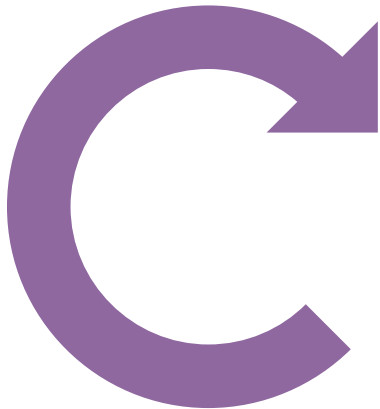
```
1 : apple  
2 : banana  
3 : strawberry
```

while



AT ALL NETWORKS

The "**while**" loop allows you to execute instructions as long as the condition is TRUE.
The loop stops when the condition is FALSE.



```
count = 0
while count < 4:
    print("Hello")
    count = count + 1
```



In a while loop, the index that is incremented does not exist by default, unlike in a for loop. If you want an index that increment, you must manage it yourself.

Exercise

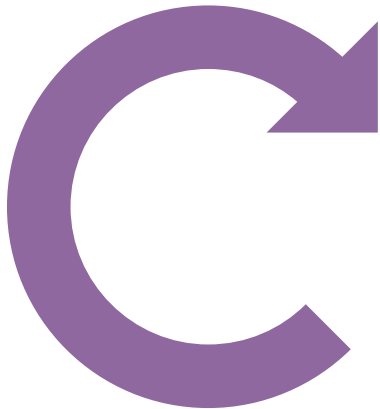
Create a game where the computer chooses a random number between 1 and 100. The user has a limited number of attempts (for example, 10) to guess the number. With each attempt, the program indicates whether the number to guess is higher or lower than the user's guess.

break



AT ALL NETWORKS

The "**break**" instruction allows you to **stop the execution** of a loop without iterating through the remaining elements.



```
while True:
    rep = input("Dites oui : ")
    if rep == "oui":
        break
```



A while loop with the condition True is an infinite loop. If no instruction is provided to stop it within the loop, the program will crash.

Exercise

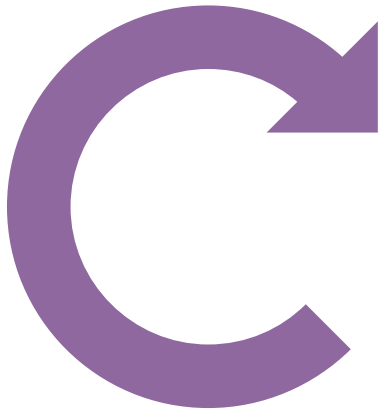
Use a loop to ask the user to provide a color, display their response, and ask if they want to continue providing a color or stop.

continue



AT ALL NETWORKS

The "**continue**" instruction allows you to **stop the current execution** of the loop and move directly to the **next iteration**.



```
for i in range(5):  
    if i == 3:  
        continue  
    print(i)
```

Exercise

Use a for loop to go through each character in a text. If the character is not a letter skip, if the character is a letter, print it in uppercase.

Exercise



AT ALL NETWORKS

Exercise

Rock – Paper – Scissors

Write a program that allows two player to play "Rock – Paper – Scissors".

Each player enter their name, and then the game begin.

Each player takes their turn, and the program determines the winner of the round and awards 1 point.

- Rock beats Scissors
- Scissors beats Paper
- Paper beats Rock

The first player to reach 3 points wins.

A tie scores no points.



AT ALL NETWORKS

Errors & Exceptions

Errors & Exceptions



AT ALL NETWORKS

Errors

Syntax errors, also known as parsing errors, are perhaps the most common kind of complaint you get while you are learning Python.

```
while True print('Hello world')
# SyntaxError: invalid syntax
```

Exceptions

Exceptions are errors detected during **execution** and are not unconditionally fatal.

```
print('2' + 2)
# TypeError: can only concatenate str(not "int") to str
```

try - except



AT ALL NETWORKS

Basic structure : try/except

The **try block runs your code**. If an exception occurs, Python looks for a matching **except** clause. If found, that block runs; otherwise, the exception propagates up.

```
try:
    x = int(input("Enter a number: "))
except ValueError:
    print("That's not a valid number!")
```

You can use the base class **Exception** or a **bare except** (not recommended, as it catches even KeyboardInterrupt and SystemExit).

```
except Exception as e:
    print(f"Something went wrong: {e}")
```

Use **as** to inspect, and access the **object** of the exception

```
except ValueError as e:
    print(f"Error details: {e}")
    print(type(e)) # <class 'ValueError'>
```



You can handle several exception types in one clause or separately

```
# As a tuple (same handling for both)
except (TypeError, ValueError):
    print("Type or value error!")

# Separately (different handling per type)
except TypeError:
    print("Wrong type")
except ValueError:
    print("Wrong value")
```

finally & else



AT ALL NETWORKS

Else

Runs only if no exception was raised in try. This is cleaner than putting success code inside try, because it avoids accidentally catching exceptions from the success code itself.

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Division by zero!")
else:
    print(f"Result is {result}") # Only runs on success
```

Finally

Always runs, whether an exception occurred or not – ideal for **cleanup**.

```
try:
    f = open("file.txt")
except FileNotFoundError:
    print("File not found")
finally:
    print("This always runs") # Good for releasing resources
```

Exemple



AT ALL NETWORKS

Exception hierarchy

All built-in exceptions inherit from **BaseException**. Most user-facing ones inherit from **Exception**.
Catching a parent class catches all its subclasses:

Full example of a try/except

```
try:
    number = int(input("Enter a number: "))
    result = 100 / number
except ValueError:
    print("Not a number!")
except ZeroDivisionError:
    print("Can't divide by zero!")
else:
    print(f"100 / {number} = {result}")
finally:
    print("Done.")
```


Resources



AT ALL NETWORKS

Documentation Python

Python documentation : <https://docs.python.org/3.12/reference/index.html>

Python conventions : <https://peps.python.org/pep-0008/>

Docstring conventions : <https://peps.python.org/pep-0257/>

Other sites

W3schools : https://www.w3schools.com/python/python_syntax.asp

GeekForGeek : <https://www.geeksforgeeks.org/python/python-programming-language-tutorial/>





AT ALL NETWORKS

We know the way!